

What is an LLM?

How a language model writes, one word at a time · video 2:05

This lesson has one big idea: a large language model does **one simple thing, over and over**. It predicts what comes next, picks something, adds it to the text, and repeats. Everything else in the series is about how the prediction works on the inside.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 9).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. One job: predict what comes next	▶ 0:08-0:30	3	☐
2. A probability for every word	▶ 0:30-0:42	4	☐
3. Pick one word and append it	▶ 0:42-0:48	4	☐
4. The loop: predict, pick, append, repeat	▶ 0:48-1:03	4	☐
5. The loop in code	▶ 1:03-1:16	3	☐
6. Inside the model: a function made of numbers	▶ 1:16-1:57	3	☐

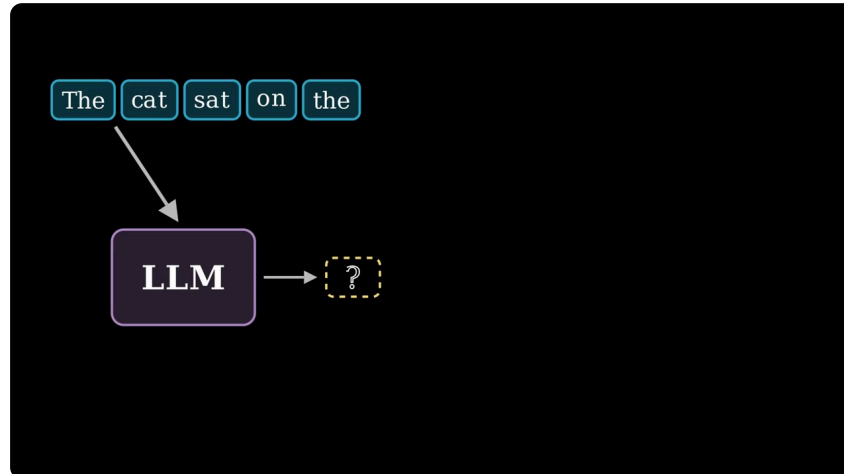
Before you start: no maths is needed for this lesson. Concept 5 reads a short Python function; if Python is new to you, Foundations F11 (NumPy in Three Minutes) helps. A note on words: the video says the model predicts the next *word*. Strictly it predicts the next *token*, a piece of a word, which is the topic of episode 2. In this guide, "word" and "token" mean the same thing.

CONCEPT 1 OF 6

One job: predict what comes next

▶ Video 0:08–0:30

You type a question and a chatbot writes an answer. It can look like the model plans the whole reply, or like someone is typing. What actually happens is simpler: the model is given **all the text so far** and answers a single question, **what comes next?**



▶ 0:29 The text so far goes into the model. Its only job is to fill the empty slot: what comes next?

KEY IDEA

An LLM (large language model) maps **the text so far** to a **guess about the next word**. Answering questions, writing code and translating are all this one operation, repeated.

Check 1: One job: predict what comes next

Answer all 3 questions before looking at the answers. Pass mark: all correct.

1.1 MULTIPLE CHOICE

You give an LLM the text *"The cat sat on the"*. What does it produce in one step?

- A. A complete paragraph continuing the story
- B. A guess about the single next word
- C. A summary of the sentence
- D. A list of grammar corrections

1.2 TRUE OR FALSE

"An LLM writes its whole reply at once, as a single output."

True ☐ False ☐

1.3 SHORT ANSWER

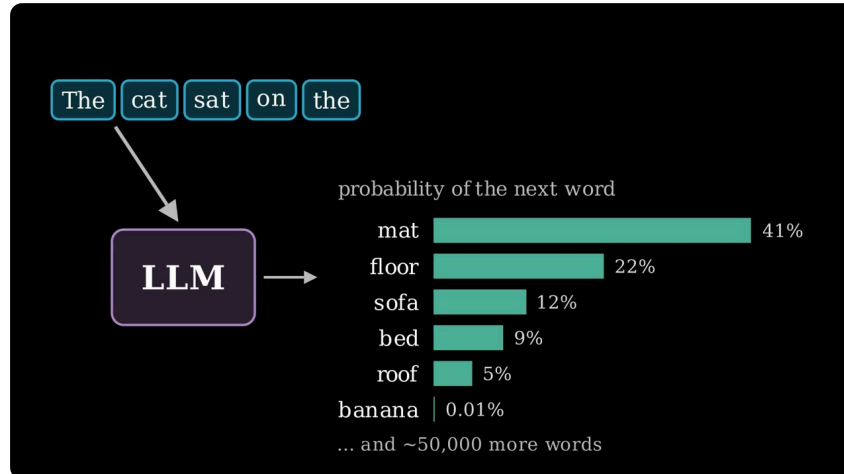
The input is *"Once upon a"*. In one sentence, describe the model's job. Which word would you expect it to rate as most likely?

CONCEPT 2 OF 6

A probability for every word

▶ Video 0:30–0:42

The model does not output just one word. It outputs a **probability for every word in its vocabulary**, tens of thousands of numbers. A probability is a number between 0 and 1 (0 % to 100 %) saying how likely each word is to come next.



▶ 0:41 The model's output for "The cat sat on the": a probability for every word it knows. Only the top few (and banana) are shown.

KEY IDEA

The output is a **probability distribution**: every value is between 0 and 1, and all of them together **add up to exactly 1** (100 %). Likely words get large values (*mat* 41 %); unlikely words get tiny values (*banana* 0.01 %), but not zero.

Check 2: A probability for every word

Answer all 4 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

In the chart, *mat* 41 %, *floor* 22 %, *sofa* 12 %, *bed* 9 % and *roof* 5 %. How much probability is left for all the other words together?

2.2 MULTIPLE CHOICE

Which list could be a probability distribution over three words?

- A. 0.5, 0.3, 0.3
- B. 0.6, 0.4, 0.0
- C. 0.7, -0.1, 0.4
- D. 0.2, 0.2, 0.2

2.3 TRUE OR FALSE

"Because *banana* is such an unlikely next word, the model gives it a probability of exactly 0."

True ☐ False ☐

2.4 CALCULATION

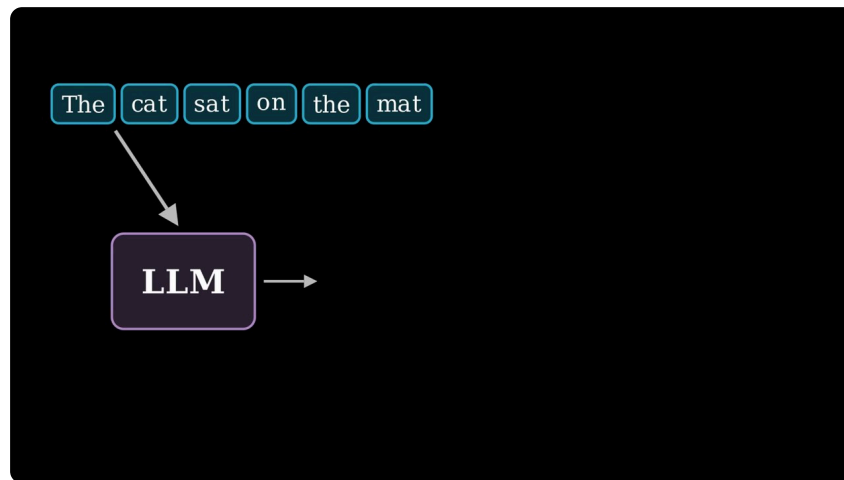
A model's vocabulary has 50,000 words. How many probabilities does it produce each time it predicts the next word?

CONCEPT 3 OF 6

Pick one word and append it

▶ Video 0:42–0:48

A distribution is not yet text. We **pick one word** from it, usually one of the likely ones, and **append** it to the end of the text.



▶ 0:47 "mat" has been picked and added to the end of the text.

There are two simple ways to pick. **Greedy**: always take the most likely word. **Sampling**: pick at random, so that each word is chosen as often as its probability says (41 % of the time *mat*, 22 % of the time *floor*, ...). Sampling gives variety; episode 10 covers the details.

KEY IDEA

Prediction gives **probabilities**; **picking** turns them into one actual word, which is then **appended** to the text.

Check 3: Pick one word and append it

Answer all 4 questions before looking at the answers. Pass mark: all correct.

3.1 SHORT ANSWER

Using the chart from concept 2, which word does **greedy** picking choose?

.....

3.2 CALCULATION

You **sample** from that same distribution 100 separate times. About how many times would you expect *floor* to be picked?

.....

3.3 SHORT ANSWER

The text is "*The cat sat on the*" and the picked word is *mat*. Write the new text.

.....

3.4 MULTIPLE CHOICE

Why pick a word that is not the single most likely one?

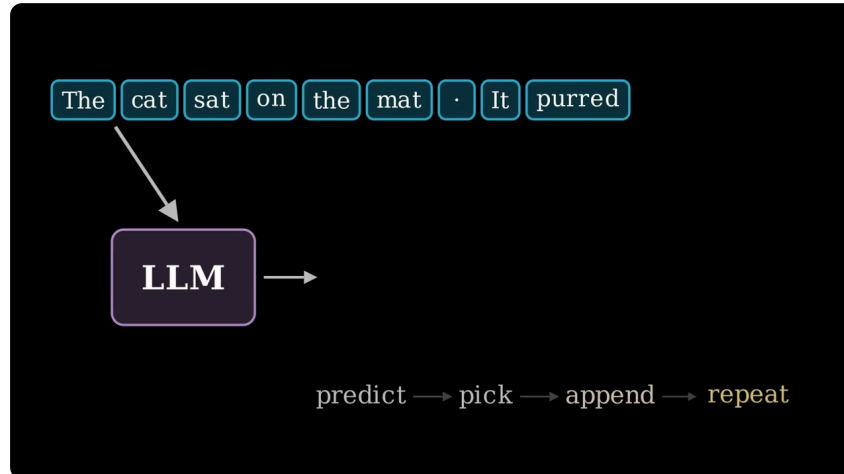
- A. Because the model is not accurate enough
- B. To add variety, so the text is not always the same or repetitive
- C. Because the probabilities must add up to 1
- D. To make generation faster

CONCEPT 4 OF 6

The loop: predict, pick, append, repeat

▶ Video 0:48–1:03

After appending, we **feed the whole text back in** and predict again: “The cat sat on the mat” → maybe a period. Then another word, and another. Each new word costs **one full run of the model**, and the model reads its own earlier words as part of its input.



▶ 1:02 The whole text goes back in for every new word. The loop that generates all text: predict → pick → append → repeat.

The loop stops when it reaches a **length limit**, or when the model predicts a special **END** token that means “I’m done”.

KEY IDEA

All text generation is one loop: **predict → pick → append → repeat**, until a length limit or an END token.

Check 4: The loop: predict, pick, append, repeat

Answer all 4 questions before looking at the answers. Pass mark: all correct.

4.1 PUT IN ORDER

Put these steps in the right order: *append · repeat · predict · pick*.

4.2 CALCULATION

Starting from “The cat sat on the”, the model generates *mat*, *.*, *It*, *purred*. (a) How many times was the model run? (b) What text was its input on the third run?

4.3 TRUE OR FALSE

“On later steps, the model reads the words it generated itself as part of its input.”

True ☐ False ☐

4.4 SHORT ANSWER

Name the two ways the loop can stop.

CONCEPT 5 OF 6

The loop in code

▶ Video 1:03-1:16

The loop fits in a few lines of Python. `tokens` is the text so far (a list of words), `model` returns the probabilities, and `sample` picks one word.

```
1 def generate(model, tokens, max_new=50):
2     for _ in range(max_new):
3         probs = model(tokens) # a probability per word
4         next_tok = sample(probs) # pick one
5         tokens.append(next_tok) # add it to the text
6         if next_tok == END: # model says it's done
7             break
8     return tokens
```

▶ 1:09 The generate function from the video: the whole loop in eight lines.

```
def generate(model, tokens, max_new=50):
    for _ in range(max_new):
        probs = model(tokens) # a probability per word
        next_tok = sample(probs) # pick one
        tokens.append(next_tok) # add it to the text
        if next_tok == END: # model says it's done
            break
    return tokens
```

KEY IDEA

One loop iteration = one new word. `max_new` is the length limit; `END` is the model's own stop signal.

Check 5: The loop in code

Answer all 3 questions before looking at the answers. Pass mark: all correct.

5.1 SHORT ANSWER

What does `max_new` control?

5.2 MULTIPLE CHOICE

Suppose the model's very first pick is `END`. Read the code carefully: what does `generate` return?

- A. The original tokens, unchanged
- B. The original tokens with `END` added at the end
- C. An empty list
- D. An error

5.3 HANDS-ON CODE

Try it yourself. Below is a toy “model” that looks only at the last word, and a version of `generate` that takes the picking function as an argument. (a) Write `greedy(probs)`, which returns the most likely word, and `sample(probs)`, which picks at random using the probabilities (hint: `random.choices`). (b) What does `generate(model, ["the"], greedy, max_new=8)` return? Why does it never reach the period? (c) With `sample`, what is the probability that the output is exactly *the mat . END*?

```
import random

END = "END"
NEXT = {
    "the": {"cat": 0.6, "mat": 0.4},
    "cat": {"sat": 1.0},
    "sat": {"on": 1.0},
    "on": {"the": 1.0},
    "mat": {"." : 1.0},
    "." : {END: 1.0},
}

def model(tokens):
    return NEXT[tokens[-1]]          # probabilities for the next word

def generate(model, tokens, pick, max_new=50):
    tokens = list(tokens)
    for _ in range(max_new):
        next_tok = pick(model(tokens))
        tokens.append(next_tok)
        if next_tok == END:
            break
    return tokens
```

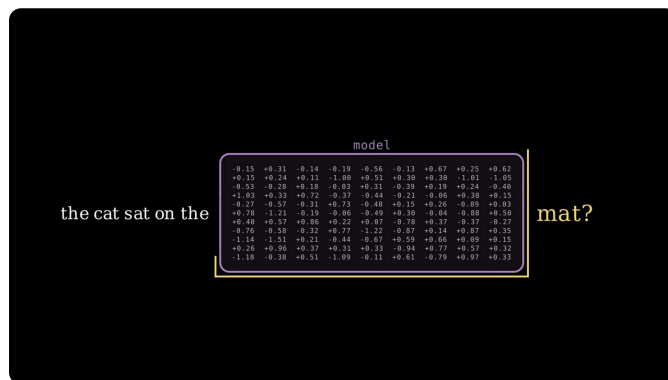
CONCEPT 6 OF 6

Inside the model: a function made of numbers

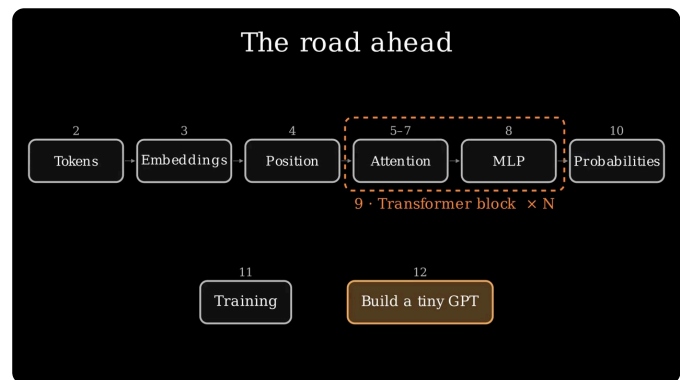
▶ Video 1:16-1:57

In the code, all the intelligence is hidden in one call: `model(tokens)`. Inside it there are no hand-written rules like “after *sat on the*, say *mat*”. There is a huge collection of numbers, called **parameters** or **weights**, and the model computes the probabilities from them. The numbers are **learned from data** during training (episode 11).

The rest of the series opens the function up, in the order the data flows through it: text is split into **tokens** (ep 2), each token becomes a vector, an **embedding** (ep 3), plus its **position** (ep 4). **Attention** lets words look at each other (eps 5–7) and the **MLP** processes each word (ep 8). Together these form a **transformer block**, stacked many times (ep 9). Finally vectors become **probabilities** again (ep 10).



▶ 1:30 Everything interesting hides inside model: a huge pile of numbers that somehow knows “mat” is likely.



▶ 1:56 The road ahead: how the series opens up the model, episode by episode.

KEY IDEA

model is a function built from millions (or billions) of learned numbers. Data flows: **tokens** → **embeddings (+ position)** → **transformer blocks (attention + MLP)** → **probabilities**.

Check 6: Inside the model: a function made of numbers

Answer all 3 questions before looking at the answers. Pass mark: all correct.

6.1 TRUE OR FALSE

“Inside an LLM there is a list of hand-written rules, such as ‘after *sat on the*, say *mat*’.”

True ☐ False ☐

6.2 PUT IN ORDER

Put these stages in the order data flows through the model: *probabilities* · *tokens* · *transformer blocks (attention + MLP)* · *embeddings*.

6.3 MULTIPLE CHOICE

Where do the model's numbers come from?

- A. Engineers write them by hand
- B. They are random and never change
- C. They are learned from lots of text during training
- D. They are copied from a dictionary

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. One job: predict what comes next · rewatch ▶ 0:08-0:30

1.1 B. One step = one prediction of the next word. Longer text comes from repeating the step.

1.2 False. The reply is produced one word at a time, each word predicted from the text so far.

1.3 Predict the next word. A well-trained model would rate “time” as by far the most likely.

2. A probability for every word · rewatch ▶ 0:30-0:42

2.1 11 %. $41 + 22 + 12 + 9 + 5 = 89$ %, and the whole distribution adds up to 100 %.

2.2 B. A adds up to 1.1; C has a negative value; D adds up to only 0.6. B is fine: a 0 is allowed, and the total is exactly 1.

2.3 False. It gets a tiny probability (0.01 % in the video), not zero. Every word in the vocabulary gets some probability.

2.4 50,000. One probability per word in the vocabulary, every single step.

3. Pick one word and append it · rewatch ▶ 0:42-0:48

3.1 mat. Greedy always takes the highest probability, and mat has 41 %.

3.2 About 22. Sampling picks each word in proportion to its probability: 22 % of 100 $\approx$ 22 (the exact count varies from run to run).

3.3 “The cat sat on the mat”. Appending adds the word at the end; nothing else changes.

3.4 B. Sampling makes the output varied. The model's probabilities are the same either way.

4. The loop: predict, pick, append, repeat · rewatch ▶ 0:48-1:03

4.1 predict → pick → append → repeat. Predict probabilities, pick a word, append it, go again.

4.2 (a) 4 times. (b) “The cat sat on the mat .” One run per new word. The third run sees the original text plus the first two generated words.

4.3 True. The whole text, including everything generated so far, goes back in each time.

4.4 A length limit, or the model producing the END token. In the code of concept 5 these are `max_new` and `next_tok == END`.

5. The loop in code · rewatch ▶ 1:03-1:16

5.1 The maximum number of new words the loop can add. The `for` loop runs at most `max_new` times, adding one word each time.

5.2 B. `tokens.append(next_tok)` runs *before* the END check, so END is appended and then the loop breaks. Real code often strips it off afterwards.

5.3 (b) ['the', 'cat', 'sat', 'on', 'the', 'cat', 'sat', 'on', 'the'] · (c) 0.4 (a) `def greedy(probs): return max(probs, key=probs.get)` and `def sample(probs): words = list(probs); return random.choices(words, weights=[probs[w] for w in words])[0]`. (b) After *the*, greedy always picks *cat* (0.6 beats 0.4), so it cycles *the cat sat on the* ... and only the `max_new` limit stops it. This is why pure greedy picking can get stuck repeating itself. (c) The first pick must be *mat* (0.4); after that `.` and `END` are certain (1.0), so $0.4 \times 1 \times 1 = 0.4$.

6. Inside the model: a function made of numbers · rewatch ► 1:16-1:57

6.1 False. It is a function of learned numbers (parameters). Nobody writes the rules.

6.2 tokens → embeddings → transformer blocks → probabilities. Text is split into tokens, tokens become vectors, blocks process them, and the result is turned back into a probability for every word.

6.3 C. They start random and training adjusts them to predict text better (episode 11).